Heaps and Priority Queues

DSA27 — Lecture 12 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 12 · Fall 2026

Where We Are

A queue that does not wait its turn

Week 7's queue is **fair**: first in, first out.

An emergency department is not fair, on purpose:

- a patient with chest pain is seen **before** one with a sprained ankle
- who arrived first matters only between equally urgent patients

We need a queue that serves the **most urgent** item: a **priority queue**.

Today

1. The priority queue ADT, and three ways to build it badly
2. The heap: a **shape** and an **order**
3. The array **is** the tree: `2i + 1`, `2i + 2`, `(i - 1) // 2`
4. Sift up (`push`) and sift down (`pop`): $O(\log n)$
5. Build-heap in $O(n)$ — the summation, and a measurement
6. The priority queue on top; top-k; heap sort revisited

The Priority Queue ADT

Serve the most urgent

Operation	Meaning
`enqueue(item, priority)`	add an item with a priority
`dequeue()`	remove and return the item with the **lowest** priority number
`peek()`	look at that item without removing it
`is_empty()`, `len(pq)`	

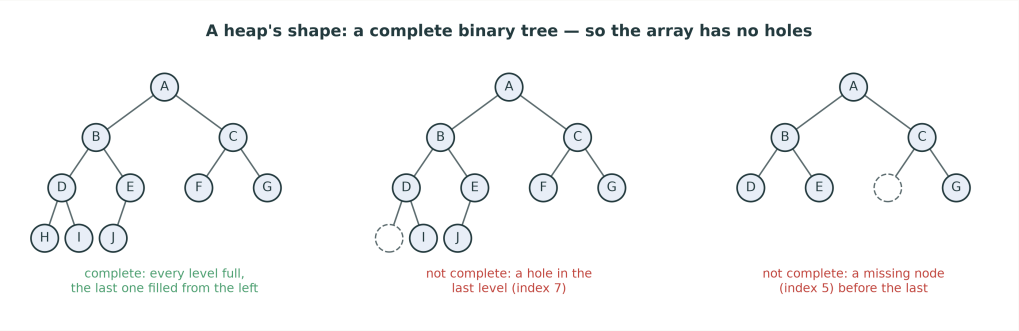
"Priority 1" means "most urgent" — as in triage, and in `dsa/heap.py`.

Three obvious designs, and what each costs

Storage	enqueue	dequeue	Why
unsorted array	$O(1)$	$O(n)$	append; then **search** for the minimum
sorted array	$O(n)$	$O(1)$	**shift** to insert in order; take from the end
sorted linked list	$O(n)$	$O(1)$	**walk** to the insertion point
binary heap	$O(\log n)$	$O(\log n)$	today

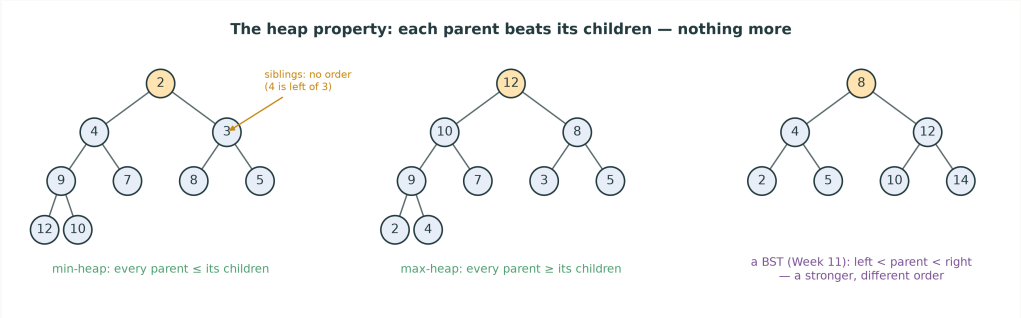
The Heap

Shape: a complete binary tree



Complete: every level is full, except perhaps the last, which is filled **from the left**, with no gaps.

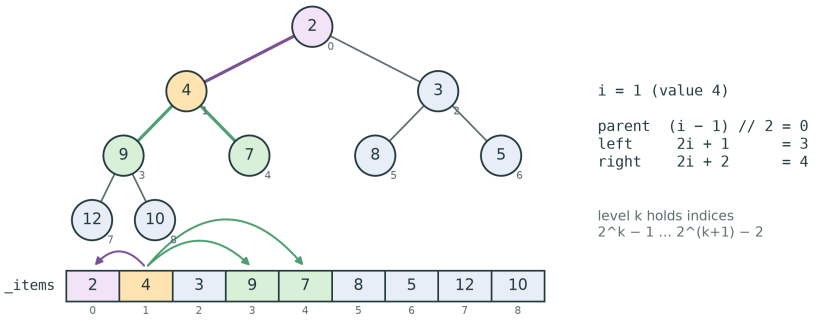
Order: the heap property



Min-heap: every parent $\leq$ its children. **Max-heap:** every parent $\geq$ its children.

So the best item is at the **root**. Siblings are **not** ordered.

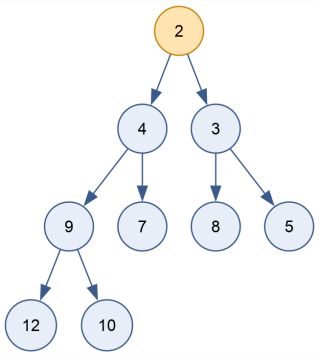
The array is the tree



Node `i`: children at **`2*i + 1`** and **`2*i + 2`**; parent at **`(i - 1) // 2`**.

See it: draw_array_as_tree

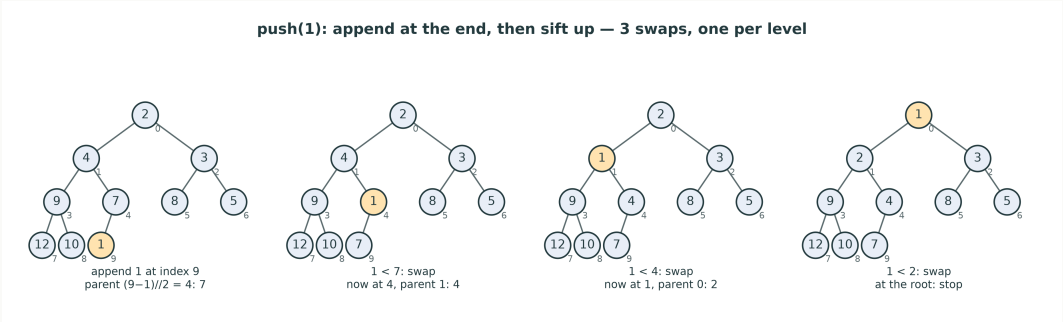
```
from viz.draw import draw_array_as_tree
draw_array_as_tree([2, 4, 3, 9, 7, 8, 5, 12, 10], highlight=0)
```



A plain list, drawn as the tree it already is. No conversion.

Sift Up: push

Append, then sift up



1. **Append** the new value at the end — the shape stays complete.
2. While it beats its **parent**, swap it with the parent.

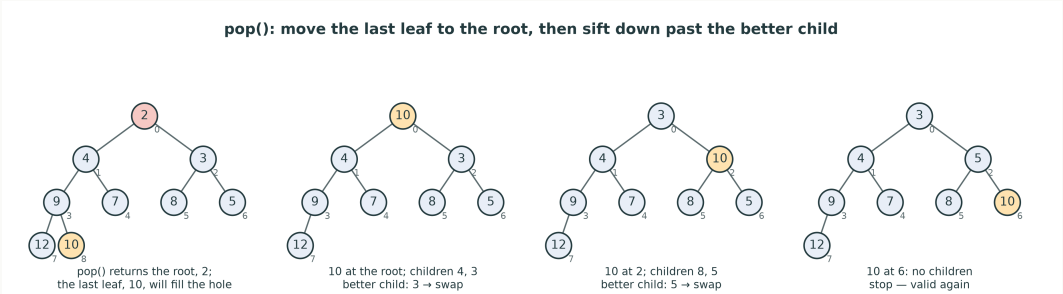
Trace: `push(1)`

Step	`1` at	parent index	parent value	Decision
append	9	(9 – 1) // 2 = 4	7	1 < 7 → swap
	4	(4 – 1) // 2 = 1	4	1 < 4 → swap
	1	(1 – 1) // 2 = 0	2	1 < 2 → swap
	0	—	—	root: stop

Result: `[1, 2, 3, 9, 4, 8, 5, 12, 10, 7]`. Three swaps: one per level.

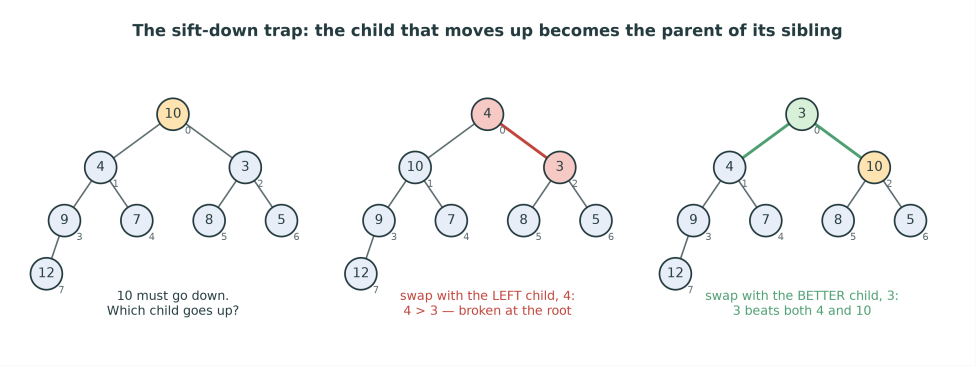
Sift Down: pop

Remove the root, fill the hole from the end



1. Keep the root — it is the answer.
2. Move the **last** item into the root; shrink the array by one.
3. While it is beaten by its **better child**, swap with that child.

The trap: swap with the better child



The child that moves up becomes the **parent of its sibling** — so it must beat the sibling too.

Both sifts are $O(\log n)$

Operation	Walks	Comparisons per level	Worst case
push $\rightarrow$ sift up	leaf to root	1	$\lfloor \log_2 n \rfloor$ swaps
pop $\rightarrow$ sift down	root to leaf	2	$2\lfloor \log_2 n \rfloor$ comparisons
peek	—	—	$O(1)$

A heap of a million items: at most **20** levels.

Build-Heap in $O(n)$

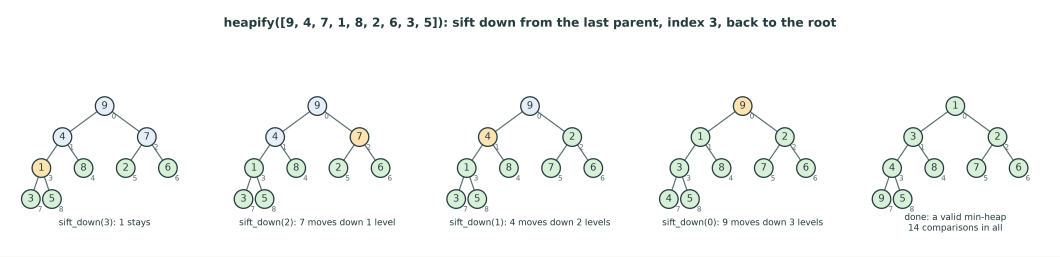
Two ways to make a heap from n values

Way 1 — n pushes. Start empty; push each value. $n \times O(\log n) = O(n \log n)$.

Way 2 — `heapify`, bottom up. Leave the values where they are. Sift **down** every parent, from the **last** parent back to the root.

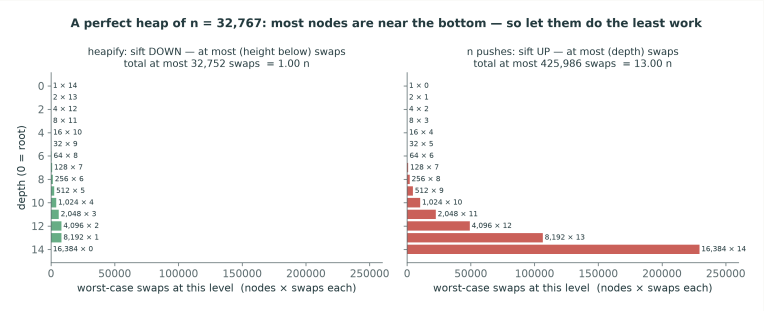
Way 2 is $O(n)$.

Trace: `heapify([9, 4, 7, 1, 8, 2, 6, 3, 5])`



n = 9, last parent = 9 // 2 – 1 = **3**. Sift down 3, 2, 1, 0.

Why heapify is $O(n)$: count the work level by level



About $n/2^{h+1}$ nodes sit at **height h**; each sifts down at most h levels:

$$\sum_{h\geq 0} \frac{n}{2^{h+1}} \cdot h \;=\; \frac{n}{2} \sum_{h\geq 0} \frac{h}{2^h} \;=\; \frac{n}{2} \cdot 2 \;=\; n$$

Measured: heapify against n pushes



The Priority Queue on Top

`PriorityQueue`: a `MinHeap` of triples

`enqueue(item, priority)` pushes the triple

$$(\text{priority},\ \text{counter},\ \text{item})$$

and adds 1 to `counter`. `dequeue()` pops a triple and returns its **item**.

- Python compares tuples **left to right**: priority first.
- `counter` is unique, so the comparison **never reaches** `item`.

Trace: triage

Operation	Heap array: `(priority, counter, item)`	Returns
enqueue A, 2	`(2,0,A)`	
enqueue C, 1	`(1,1,C) (2,0,A)`	
enqueue K, 3	`(1,1,C) (2,0,A) (3,2,K)`	
enqueue L, 1	`(1,1,C) (1,3,L) (3,2,K) (2,0,A)`	
dequeue	`(1,3,L) (2,0,A) (3,2,K)`	C
dequeue	`(2,0,A) (3,2,K)`	L

A broken arm · C chest pain · K sprained ankle · L allergic reaction

Where priority queues are used

- **Scheduling:** an operating system runs the highest-priority ready process; a print server, a build server, a job queue.
- **Triage:** emergency departments; bug trackers (“P1 before P3”).
- **Event simulation:** the next event is the one with the smallest time.
- **Top-k:** the k largest of n values, in $O(n \log k)$.
- **Graph algorithms:** Dijkstra’s shortest paths, Prim’s spanning tree, Huffman coding — *beyond this course’s bylaw*.

Top-k: keep a small heap, not a big one

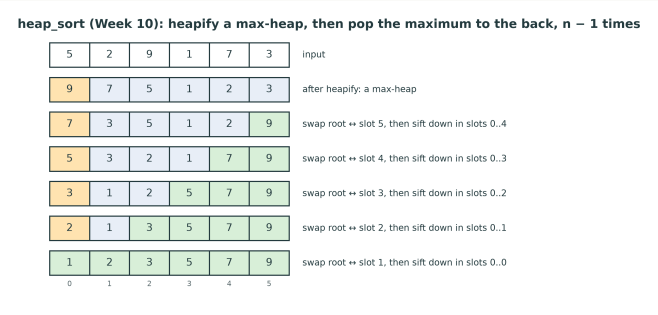
The k **largest** of n values:

1. Keep a **min**-heap of at most k values.
2. For each value: if the heap has fewer than k, push it. Otherwise, if the value beats the heap's **smallest** (its root), pop the root and push the value.
3. The heap now holds the k largest.

$O(n \log k)$ time, $O(k)$ space. Sorting everything: $O(n \log n)$ and $O(n)$.

Heap Sort, Revisited

Week 10 was a heap in disguise



1. `heapify` the array as a **max**-heap: $O(n)$.
2. Repeat n – 1 times: swap the root (the maximum) with the **last** item of the heap; the heap shrinks by one; sift the new root down: $O(\log n)$.

$O(n \log n)$ always, $O(1)$ extra space, **not stable**.

This Week

Exercises: `dsa/heap.py`

Method	Target	The trap
`_sift_up`	$O(\log n)$	parent is `(i - 1) // 2`; stop at the root
`_sift_down`	$O(\log n)$	the **better** child; a missing right child
`push`, `peek`	$O(\log n)$, $O(1)$	`IndexError` on an empty `peek`
`pop`	$O(\log n)$	last item to the root; the one-item heap
`heapify`	$O(n)$	from `n // 2 - 1` **down** to 0 — not n pushes
`is_valid`	$O(n)$	check **every** child against its parent
`PriorityQueue`	$O(\log n)$	the triple; return the **item**

```
pytest tests/test_heap.py -v
```

Homework 12 — before Lecture 13

1. **Implement** `dsa/heap.py` until all 28 tests pass.
2. **Trace** by hand, as trees and as arrays: push 6, 2, 8, 1, 5, 3 into an empty `MinHeap`; then pop twice.
3. **Heapify** `[3, 9, 2, 1, 4, 5, 8, 7, 6, 0]` by hand. Count the swaps.
4. **Measure** heapify against n pushes on descending input in `notebooks/12-heaps.ipynb`, and count `_beats` calls with a subclass.

Practise online

- **Week 12 quiz**: 22 multiple-choice questions, every answer explained
- **Written practice** on the same page: 5 questions. Write, then mark yourself against the marking points
- In Arabic: week 12

Summary

Seven things to keep

1. A priority queue serves the **most urgent** item; a heap gives $O(\log n)$ enqueue **and** dequeue.
2. **Shape:** a complete binary tree — height $\lfloor \log_2 n \rfloor$.
3. **Order:** every parent beats its children; siblings are unordered.
4. The array **is** the tree: children `2i + 1`, `2i + 2`; parent `(i - 1) // 2`.
5. `push` = append + **sift up**; pop = last to root + **sift down** past the **better** child.
6. `heapify` sifts down from `n // 2 - 1` to 0: $O(n)$, because most nodes are near the bottom.
7. `PriorityQueue` pushes `(priority, counter, item)`: the counter breaks ties — first come, first served.

Next

Week 13 — Hash tables. From $O(\log n)$ to $O(1)$ — **on average**. What "average" promises, and what breaks it.